FlowDistill: Autocorrelation-Aware Traffic Flow Prediction via Distillation from LLMs

Chenyang Yu, Xinpeng Xie, Chenxi Qiu and Yan Huang
{chenyangyu,xinpengxie}@my.unt.edu,{chenxi.qiu,yan.huang}@unt.edu
Department of Computer Science and Engineering, University of North Texas
Denton, Texas, USA

Abstract

Accurate traffic flow prediction is vital for optimizing urban mobility, yet it remains difficult in many cities due to complex spatial-temporal dependencies and limited high-quality training data. While deep graph-based models demonstrate strong predictive power, their performance often comes at the cost of high computational overhead and substantial data requirements. Recent Large Language Models (LLMs) exhibit remarkable generalization capabilities; however, they are not inherently designed for directly modeling continuous spatial-temporal autocorrelation. To address these challenges, we introduce *FlowDistill*, a lightweight framework where a spatial-temporal adapted LLM teacher guides a compact multilayer perceptron (MLP) student through knowledge distillation. We introduce explicit mechanisms within *FlowDistill* to capture and preserve spatial-temporal autocorrelation throughout the distillation process. As a result, *FlowDistill* outperforms state-of-the-art models in prediction accuracy, even when training data is scarce. Furthermore, *FlowDistill* requires significantly less training data, achieves lower memory usage, and demonstrates robust performance in zero-shot transfer scenarios, underscoring its effectiveness and suitability for real-world urban deployment.

ACM Reference Format:
Chenyang Yu, Xinpeng Xie, Chenxi Qiu and Yan Huang. 2025. FlowDistill: Autocorrelation-Aware Traffic Flow Prediction via Distillation from LLMs . In *The 33rd ACM International Conference on Advances in Geographic Information Systems (SIGSPATIAL '25), November 3–6, 2025, Minneapolis, MN, USA.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3748636.3762776

1 Introduction

Traffic flow prediction serves as a foundation for urban transportation systems. It enables smarter urban planning, optimizes resource allocation, and improves traffic management and mobility services [31]. In the past few years, *graph-based methods* have become dominant in traffic flow prediction tasks [3, 9, 20, 33]. These methods leverage graph structures to model spatial relationships while effectively capturing temporal dynamics. However, as illustrated in Figure 1, graph-based methods (categorized as "Graph (w/o KD)") depend heavily on sufficient training datasets to maintain high prediction accuracy. This limits their effectiveness [22], especially in *limited training data* scenarios [29].

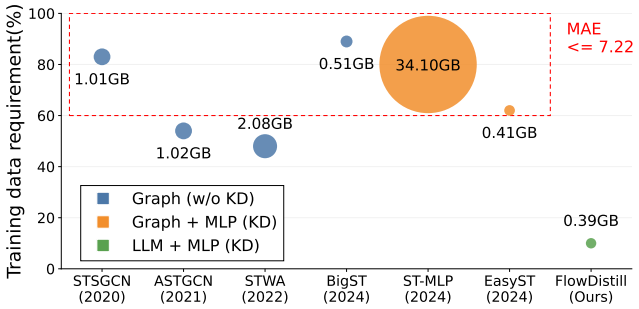


Figure 1: The comparison of *memory cost* and *training data required* to achieve the same performance (i.e., MAE = 7.22 on the NYC dataset). Note that STSGCN, BigST, ST-MLP, and EasyST fail to reach MAE = 7.22 even with the maximum training data proportion. The detailed explanation of the experimental results related to this figure can be found in Section 5.3. Here, memory cost is represented by the bubble size.

Meanwhile, the recent fast advances in *large language models (LLMs)* [1] have opened new opportunities for traffic flow prediction. Unlike graph-based methods, which rely on predefined spatial structures and abundant labeled data to learn relationships, LLMs benefit from a broad base of pre-trained knowledge gathered from large and diverse corpora [8]. This foundational knowledge enables them to identify high-level patterns and contextual cues with minimal additional data, making LLMs particularly well-suited to traffic flow prediction tasks with limited training data. Notably, existing models such as UrbanGPT [18] and LLM-COD [27] have leveraged these strengths, demonstrating promising prediction accuracy even in scenarios with sparse or incomplete training datasets.

Despite their strengths, LLMs, with their billions of parameters, require significant computational resources. This makes their deployment on mobile or edge devices particularly challenging due to the inherent resource constraints of such environments. One potential solution to solve it could be *knowledge distillation (KD)* [12]. KD enables the transfer of knowledge from a high-capacity teacher model to a lightweight student model, reducing computational demands while maintaining predictive accuracy. Recent works, such as EasyST [23], SSGNN [14], and ST-MLP [29], have demonstrated the effectiveness of KD in improving inference speed and scalability for spatial-temporal tasks. For example, EasyST [23] employs techniques like spatial-temporal bottlenecks and teacher-bounded loss functions to mitigate noise and align predictions. However, these

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGSPATIAL '25, Minneapolis, MN, USA
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-2086-4/2025/11
https://doi.org/10.1145/3748636.3762776

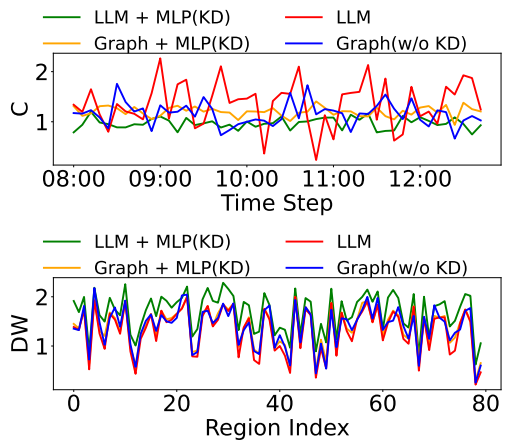


Figure 2: The comparison of *Geary'C* [7] statistics in five hours and *Durbin–Watson Test* [4] per region for the best performing model from four categories: LLM + MLP (FlowDistill), LLM (Qwen3), Graph + MLP (EasyST) and Graph (STWA). A value of $DW \approx 2$ for the Durbin-Watson statistic or $C \approx 1$ for Geary's C indicates that the autocorrelation is captured. The detailed explanation of the experimental results related to this figure can be found in Section 5.6.

existing approaches rely on graph-based teacher models and have not explored KD for traffic flow prediction using large language models (LLMs). In contrast, our work introduces an LLM-based teacher model, leveraging its advanced contextual understanding to address both scalability challenges and predictive accuracy in spatial-temporal KD frameworks.

Additionally, as Zheng et al. [32] note, LLMs are inherently designed for language understanding and struggle to capture temporal dependencies. A primary reason for this shortfall is the pseudo-alignment between time series data modality and language-oriented LLM architectures, which prevents meaningful transfer of language knowledge to time series tasks. Furthermore, as Figure 2 illustrates, compared with graph-based and KD models, the LLM (Qwen3) alone exhibits the greatest variability for Geary's C statistic (denoted as C), and its Durbin-Watson value deviates the most from 2, indicating its difficulty in capturing spatial and temporal autocorrelations. To overcome this limitation, we incorporate a spatial-temporal autocorrelation-aware loss during knowledge distillation to recapture these dependencies.

In this work, we use knowledge distillation to transfer the LLM's knowledge to a lightweight student model, reducing computational demands while maintaining predictive accuracy. Because many real-world phenomena depend on space and time, incorporating the spatial-temporal autocorrelation-aware loss into this process is essential to avoid biased estimates, improve generalization, and prevent suboptimal predictions. Our key contributions are summarized as follows:

- We propose an innovative teacher-student architecture that leverages a fine-tuned spatial-temporal LLM as the teacher and a lightweight MLP as the student. This framework bridges the gap between the high generalization power of LLMs and the computational efficiency of MLPs, enabling efficient traffic flow prediction under limited training data.
- Our framework leverages LLM instruction tuning and explicitly models spatial–temporal autocorrelation in knowledge distillation. By grounding predictions in the intrinsic spatial-temporal relationships present in real-world data, the model improves generalization with limited training data, and avoids performance degradation caused by ignoring these dependencies.
- Extensive experiments on large scale datasets demonstrate that our approach captures spatial-temporal autocorrelation and significantly outperforms all state-of-the-art Graph and Graph-MLP models in terms of both predictive accuracy and computational efficiency.

2 Related Work

2.1 Traffic Flow Prediction

Traffic flow prediction plays a critical role in urban planning, traffic management, and mobility optimization [5, 31]. The advent of deep learning introduced the use of Convolutional Neural Networks (CNNs) to capture spatial patterns, typically by modeling traffic networks as grid-like structures [11, 15, 30]. However, such representations fail to reflect the underlying graph-based topology and complex spatial dependencies of real-world traffic networks. Consequently, Graph Neural Networks (GNNs) have become dominant due to their fundamental ability to operate on graph-structured data [6, 24–26].

More recently, Large Language Models (LLMs) have emerged as a promising direction, showing potential in capturing high-level spatial-temporal patterns due to their extensive pre-training. UrbanGPT [18] developes a spatial-temporal LLM framework employing specific encoders and instruction tuning methods to enhance LLM comprehension of spatial-temporal dependencies. LLM-COD [27] leverages the knowledge of LLMs to infuse understanding of origin and destination pairs. However, the massive scale of LLMs poses significant deployment challenges. Moreover, their primary training on textual data may limit their capacity to model the distinctive autocorrelation characteristics of time-series data, which often display stronger and more regular temporal patterns than natural language [21, 32].

2.2 Light-Weight Model for Traffic Flow Prediction

Due to the model complexity, GNN based models need extensive training time. There are some light-weight methods for traffic flow prediction from the perspective of linearity. For example, BigST [9] build a linearized global spatial convolution network to adaptively distill time-varying graph structures.

Knowledge distillation (KD) [12] is a widely used technique for model compression. Recent efforts have explored distilling knowledge from complex Spatial-Temporal Graph Neural Networks (STGNNs) into lightweight Multi-Layer Perceptrons (MLPs) to improve efficiency since MLPs have excellent nonlinear fitting ability by relying on multiple linear layers and nonlinear activation functions. For example, ST-MLP [29] accelerates real-time traffic flow inference by transferring knowledge from STGNNs into MLPs using

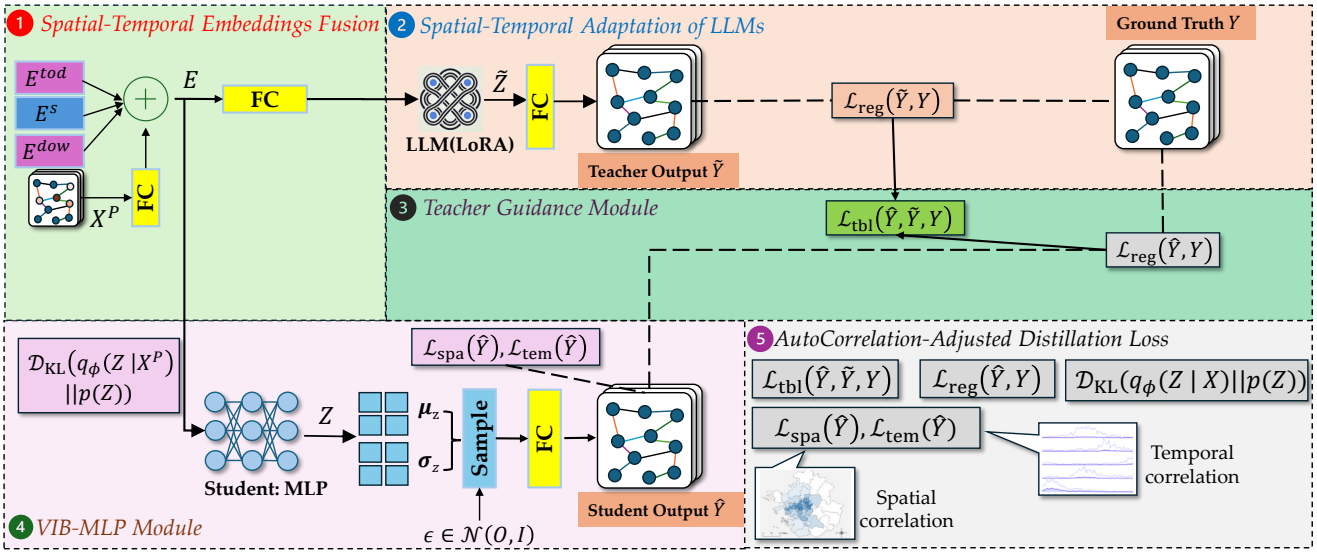


Figure 3: Overall model framework. ① Spatial-Temporal Embedding Fusion. ② Spatial-Temporal Adaptation of LLMs. ③ Teacher Guidance Module. ④ Variational Information Bottleneck Guided MLP (VIB-MLP) Module: The shared spatial-temporal embeddings E are processed through an MLP to derive the latent variables μ_Z and σ_Z^2. Using the reparameterization trick, the latent representation Z is sampled and passed through a fully connected layer to generate the final prediction $\hat{Y}$. ⑤ Autocorrelation-Adjusted Distillation Loss.

specialized spatial-temporal mixing layers. Similarly, EasyST [22] enhances scalability and generalization by introducing information bottlenecks and a teacher-bounded loss during the distillation process. These methods primarily employ GNNs as teacher models to produce efficient student models for spatial-temporal prediction tasks. While these approaches demonstrate the value of KD in traffic forecasting, they have yet to explore the potential of using LLMs as teacher models. Furthermore, it remains unclear how to enable LLMs to capture and preserve the spatial and temporal autocorrelation during knowledge distillation. Our work, FlowDistill, bridges this gap. To the best of our knowledge, FlowDistill is the first framework that distills knowledge from a fine-tuned spatial-temporal LLM into a lightweight MLP student model while effectively capturing spatial-temporal dependencies during the distillation process.

3 Preliminary

We first introduced the main definitions used in the paper. Frequently used notations throughout the paper are summarized in Table 1.

Definition 1 (Regions). *We divide a city into N regions and monitor traffic status in each region. Let $\mathcal{V} = \{v_1, \ldots, v_N\}$ denote the set of regions, where each v_i represents one region.*

Definition 2 (Road Network). *We model the spatial relationships among regions as an undirected graph $\mathcal{G} = (\mathcal{V}, \mathbf{A})$, where $\mathbf{A} \in \mathbb{R}^{N \times N}$ is a weighted adjacency matrix. Each entry in $\mathbf{A}$ encodes the spatial dependency between a pair of regions. The entire observation period is divided into T discrete time intervals, indexed by $t = 1, \ldots, T$.*

Definition 3 (Traffic Flow). *Traffic flow is represented as a two-dimensional tensor $\mathbf{X} = (X_1, \ldots, X_T) \in \mathbb{R}^{N \times T}$. For each time interval t, $X_t = (x_t^{(1)}, \ldots, x_t^{(N)})$ denotes the traffic flow values of all N regions, where $x_t^{(s)}$ is the traffic flow value of region s at time t.*

Definition 4 (Traffic Flow Prediction). *Given historical traffic flow observations from the past H_{in} time intervals, the goal of traffic flow prediction is to forecast the traffic flow for the next H_{out} time intervals. Formally,*

$$\hat{Y}_{t+1}, \ldots, \hat{Y}_{t+H_{out}} = f(X_{t-H_{in}+1}, \ldots, X_t), \tag{1}$$

where $f(\cdot)$ denotes the prediction model and $\hat{Y}_{t+h}$ is the predicted traffic flow at future time step $t + h$.

4 Methodology

In this section, we present our FlowDistill framework. Our goal is to distill valuable knowledge from a large language model (teacher) to a lightweight MLP (student), while explicitly capturing spatial and temporal autocorrelations in traffic data. As illustrated in Figure 3, our FlowDistill framework consists of five primary components: ① *Spatial-Temporal Embedding Fusion*, ② *Spatial-Temporal Adaptation of LLMs*, ③ *Teacher Guidance Module*, ④ *Variational Information Bottleneck Guided MLP (VIB-MLP) Module*, and ⑤ *Autocorrelation-Adjusted Distillation Loss*.

4.1 ① Spatial-Temporal Embeddings Fusion

To capture the dynamic variability of traffic across both space and time, we follow the identity embedding methodology from STID [19]. We set the embedding dimension to d.

Table 1: Main notations and their descriptions

Symbol	Description
N	The total number of regions
T	The total length of the timesteps
$x_t^{(s)}$	Traffic flow value of the s-th region at the t-th time interval
$X_t = (x_t^{(1)}, ..., x_t^{(N)})$	Traffic flow values of all N regions at the t-th time interval.
$y_t^{(s)}$	The ground truth flow for region s at time t
$\hat{y}_t^{(s)}/\tilde{y}_t^{(s)}$	The predicted flow of student/teacher model for region s at time t
$H_{\text{in}}/H_{\text{out}}$	The length of the input/output time window
$\mathbf{X}^P = \{X_{t-H_{\text{in}}+1}, \ldots, X_t\} \in \mathbb{R}^{N\times H_{\text{in}}}$	Origin Traffic flow Input
$\mathbf{Y} \in \mathbb{R}^{N\times H_{\text{out}}}$	Ground Truth flow
$\hat{\mathbf{Y}} \in \mathbb{R}^{N\times H_{\text{out}}}$	Predicted flow of student model
$\tilde{\mathbf{Y}} \in \mathbb{R}^{N\times H_{\text{out}}}$	Predicted flow of teacher model
$\mathbf{E}^s \in \mathbb{R}^{N\times d}$	Learnable spatial context
$\mathbf{E}^{\text{tod}} \in \mathbb{R}^{N\times d}$	Learnable "time of day" context
$\mathbf{E}^{\text{dow}} \in \mathbb{R}^{N\times d}$	Learnable "day of week" context

The spatial context, representing region-specific features, is embedded as $\mathbf{E}^s \in \mathbb{R}^{N\times d}$, where N is the number of regions. Temporal context is encoded separately for the time of day and day of the week as $\mathbf{X}^{\text{tod}} \in \mathbb{R}^{N\times T_1}$ and $\mathbf{X}^{\text{dow}} \in \mathbb{R}^{N\times T_2}$, where T_1 and T_2 denote the number of intervals within a day and the number of days in a week, respectively. This design captures the distinct temporal patterns in traffic data, such as periodic variations caused by daily or weekly routines. We project these two temporal embeddings using learnable weights $\mathbf{W}^{\text{tod}} \in \mathbb{R}^{T_1\times d}$ and $\mathbf{W}^{\text{dow}} \in \mathbb{R}^{T_2\times d}$:

$$\mathbf{E}^{\text{tod}} = \mathbf{W}^{\text{tod}} \cdot \mathbf{X}^{\text{tod}}, \quad \mathbf{E}^{\text{dow}} = \mathbf{W}^{\text{dow}} \cdot \mathbf{X}^{\text{dow}} \tag{2}$$

$$\mathbf{E}^{\text{T}} = \mathbf{E}^{\text{tod}} + \mathbf{E}^{\text{dow}} \in \mathbb{R}^{N\times d}. \tag{3}$$

We group the input traffic flow into a patch $\mathbf{X}^P = \{X_{t-H_{\text{in}}+1}, \ldots, X_t\} \in \mathbb{R}^{N\times H_{\text{in}}}$ and aggregate it with the spatial and temporal embeddings. A fully connected layer (FC) transforms the concatenated embeddings to the target dimension:

$$\mathbf{E} = \text{FC}(\mathbf{X}^P) \,||\, \mathbf{E}^s \,||\, \mathbf{E}^{\text{tod}} \,||\, \mathbf{E}^{\text{dow}} \in \mathbb{R}^{N\times 3d} \tag{4}$$

Finally, to match the input dimension required by the LLM, we project the fused embedding:

$$\mathbf{E}^{\text{llm}} = \text{FC}(\mathbf{E}). \tag{5}$$

4.2 ② Spatial-Temporal Adaptation of LLMs

Although LLMs have strong sequence modeling and semantic reasoning capabilities, directly applying them to spatial-temporal forecasting remains challenging. Traffic data are numerical, highly structured, and governed by spatial dependencies and temporal periodicity, which differ substantially from the linguistic patterns captured during standard LLM pretraining. To bridge this modality gap, we design a structured prompt that converts traffic forecasting into an instruction-following task. The prompt consists of four components: task description, historical traffic pattern and prediction target, region-level semantic context, and spatial-temporal embedding injection.

The region-level semantic context is represented by POI information. POIs indicate the functional characteristics of each region, such as commercial, residential, educational, medical, or recreational areas. These functions are closely related to traffic demand; for example, schools may induce morning and afternoon peaks, while restaurants and commercial areas may lead to higher flows during lunch, evening, or weekends. We also provide timestamps in a natural language format including date, time, weekday, and sampling interval. This helps the LLM capture distinct temporal patterns in traffic data, such as periodic variations caused by daily or weekly routines.

In addition to textual context, we inject the fused spatial-temporal embeddings into the LLM through special tokens. Specifically, the historical traffic representation obtained from the embedding fusion layer is inserted as `<ST_HIS>`, while the LLM is instructed to generate predictive tokens in the form of `<ST_PRE>`. This design allows the LLM to jointly leverage textual semantic information and continuous traffic representations.

To adapt the LLM efficiently while preserving its pretrained knowledge, we employ the Low-Rank Adaptation (LoRA) [13] strategy. LoRA reduces the number of trainable parameters and computational requirements, while still allowing the model to learn task-specific spatial-temporal dependencies effectively.

Formally, for a pre-trained weight matrix $\mathbf{W} \in \mathbb{R}^{d\times k}$, LoRA updates the weights by learning a low-rank approximation:

$$\mathbf{W}' = \mathbf{W} + \Delta\mathbf{W} = \mathbf{W} + \mathbf{BA}, \tag{6}$$

where $\mathbf{B} \in \mathbb{R}^{d\times r}$ and $\mathbf{A} \in \mathbb{R}^{r\times k}$ are the trainable decomposition matrices, and the rank $r \ll \min(d, k)$.

Given the structured prompt $\mathcal{P}$ and the injected spatial-temporal embedding $\mathbf{E}^{\text{llm}}$, the LoRA-adapted LLM produces the hidden representation:

$$\tilde{\mathbf{Z}} = \text{LLM}_{\theta_f, \theta_{\text{LoRA}}}\left(\mathcal{P}, \mathbf{E}^{\text{llm}}\right), \tag{7}$$

where θ_f denotes the frozen pre-trained LLM parameters and θ_{LoRA} denotes the trainable LoRA parameters. The output representation is then projected into the traffic prediction space using a fully connected layer:

$$\tilde{\mathbf{Y}} = \text{FC}(\tilde{\mathbf{Z}}). \tag{8}$$

The overall process is illustrated in Figure 3, and the prompt template is shown in Table 2.

4.3 ③ Teacher Guidance Module

To effectively guide the knowledge distillation process for regression tasks, we apply the teacher's bounded loss [22], which ensures that the student model benefits from the teacher LLM's guidance only when the teacher's predictions provide a meaningful improvement over the student's predictions. The loss is defined as:

$$\mathcal{L}_{\text{tbl}}(\hat{Y}, \tilde{Y}, Y) = \begin{cases} \mathcal{L}_{\text{reg}}(\hat{Y}, Y) & \mathcal{L}_{\text{reg}}(\tilde{Y}, Y) - \mathcal{L}_{\text{reg}}(\hat{Y}, Y) < \delta \\ 0 & \text{otherwise.} \end{cases} \tag{9}$$

Table 2: Instruction template for inflow and outflow prediction

Component	Prompt Content
Task Description	You are a traffic forecasting assistant for New York City. Given historical taxi inflow and outflow records, region-level POI information, and the corresponding timestamps, predict future taxi inflow and outflow for the specified regions.
Historical Traffic Pattern and Prediction Target	Given historical taxi flow data over H_{in} time steps across N regions, the recorded taxi inflows are [...], and the recorded taxi outflows are [...]. The historical records cover the period from *January 11, 2020, 09:30, Wednesday* to *January 11, 2020, 15:00, Wednesday*, with data sampled every 30 minutes. Predict taxi inflow and outflow for the next H_{out} time steps, from *January 2, 2020, 15:30, Thursday* to *January 3, 2020, 21:00, Friday*, with data sampled every 30 minutes.
Region Semantic Context	The region contains POIs within a four-kilometer radius, including cafes, schools, hardware stores, supermarkets, pharmacies, restaurants, clothing stores, department stores, lodging places, and medical facilities. These POIs describe the functional characteristics of the region and provide semantic cues for traffic demand.
Spatial-Temporal Embedding Injection	The embedding fusion layer encodes the historical traffic observations as spatial-temporal tokens `<ST_HIS>`, where the tokens represent historical inflow and outflow patterns. Based on the historical traffic pattern, timestamp information, and region semantic context, generate predictive tokens for regression in the form of `<ST_PRE>`.

where $\hat{Y}$ represents the output of the student model, $\tilde{Y}$ denotes the output of the teacher model, Y is the ground truth, and $\mathcal{L}_{\text{reg}}$ is the regression loss (the specific form is detailed in the experimental setting part). The parameter δ is a predefined threshold. If the teacher's performance is only marginally better than the student's or worse, the loss is set to zero to prevent unnecessary penalties and to encourage the student model to rely more on its learning capacity. This prevents potential interference from noisy or suboptimal teacher outputs, thereby improving the robustness and convergence of the student model during training.

4.4 ④ Variational Information Bottleneck Guided MLP (VIB-MLP) Module

As shown in the lower right corner of Figure 3, the student model directly receives the fused spatial-temporal embeddings $\mathbf{E}$ (generated by the shared embedding fusion layer). These embeddings are processed by the student MLP to model the latent distribution $\mathbf{Z}$. Using the reparameterization trick [16], samples are drawn from this distribution, and the final prediction $\hat{Y}$ is produced through a fully connected (FC) layer.

4.4.1 MLP Student. We employ an MLP as the student model. As illustrated in Figure 3, the hidden representation of the student model is denoted as $\mathbf{Z} \in \mathbb{R}^{N \times H_{\text{in}} \times d}$. To achieve an optimal trade-off between predictive accuracy and model complexity, we leverage the information bottleneck (IB) principle [2]. The IB principle posits that a good representation should retain maximal relevant information about the target variable Y while minimizing redundant information about the input X. This trade-off can be formalized as the following optimization problem:

$$\min_{p(Z|X)} \left[I(X;Z) - \lambda_{\text{KL}} I(Z;Y) \right] \tag{10}$$

where $I(\cdot;\cdot)$ denotes the mutual information between two variables, $\lambda_{\text{KL}} > 0$ is a trade-off parameter balancing the level of compression and predictive performance, and $p(Z|X)$ represents the conditional probability distribution of Z given X.

Following the VIB approach [2], and assuming that $q_\phi(Z|X)$ approximates the true posterior $p(Z|X)$, Equation (10) can be reformulated as:

$$\min_{p(Z|X)} -\mathbb{E}_{\epsilon \sim p(\epsilon)} [\log q_\phi(Y|Z)] + \lambda_{\text{KL}} \mathcal{D}_{\text{KL}}(q_\phi(Z|X) \parallel p(Z)). \tag{11}$$

Here, $\epsilon \sim \mathcal{N}(0, I)$ represents an auxiliary Gaussian noise variable, which is introduced by the reparameterization trick to ensure differentiability during training. $\mathcal{D}_{\text{KL}}$ denotes the Kullback–Leibler (KL) divergence. Equation (11) enables the noise variable to remain independent of the model parameters, allowing optimization via backpropagation [2].

To instantiate this framework, the VIB approach can be viewed as a variational encoder-decoder analogous to the Variational Autoencoder (VAE) [16]. The latent encoding distribution $p(Z)$ can be treated as a latent prior and the variational decoding distribution $q_\phi(Y|Z)$ acts as a decoder.

4.4.2 Encoder. We let the prior distribution $p(Z)$ to be modeled as a fixed d-dimensional spherical Gaussian. To realize the reparameterization process, the output latent representation Z is decomposed into two components: the mean ($\boldsymbol{\mu}_Z$) and the variance ($\boldsymbol{\sigma}_Z^2$). Specifically, the first K-dimensional outputs represent $\boldsymbol{\mu}_Z$, while the remaining K-dimensional outputs correspond to $\boldsymbol{\sigma}_Z^2$. The parameter K defines the bottleneck size, which controls the level of compression in the latent space. To ensure non-negativity of the variance, a softplus transformation is applied.

4.4.3 Decoder. The posterior distribution $q_\phi(Z|X)$, modeled by the MLP, represents the learned distribution of Z given the input X, and is expressed as:

$$q_\phi(Z|X) = \mathcal{N}(\boldsymbol{\mu}_Z, \boldsymbol{\sigma}_Z^2). \tag{12}$$

The latent variable Z is obtained by the reparameterization trick:

$$Z = \boldsymbol{\mu}_Z + \boldsymbol{\sigma}_Z^2 \odot \epsilon, \tag{13}$$

where $\epsilon \sim \mathcal{N}(0, I)$ and the element-wise product $\boldsymbol{\sigma}_Z^2 \odot \epsilon$ introduces stochasticity into the sampling process.

A crucial aspect of the training process is minimizing the KL divergence between the learned posterior $q_\phi(Z|X)$ and the prior $p(Z)$. For Gaussian distributions, the KL divergence can be computed as:

$$\begin{aligned} D_{\mathrm{KL}}(q_\phi(Z|X) \parallel p(Z)) &= \int q_\phi(Z|X) \log \frac{q_\phi(Z|X)}{p(Z)}\, dZ \\ &= \sum_{i=1}^{N \times H_{\mathrm{in}} \times d} \frac{1}{2} \left(\log \frac{1}{\sigma_i^2} - 1 + \sigma_i^2 + \mu_i^2 \right), \end{aligned} \tag{14}$$

of which the detailed derivation is given in Appendix of the supplementary file.

For regression tasks involving continuous predictions, the first term in Equation (11), which corresponds to the expected negative log-likelihood, is replaced by the regression loss. Specifically, the prediction $\hat{\mathbf{Y}}$ is obtained based on the sampled latent variable $\mathbf{Z}$. The fully connected layer takes $\mathbf{Z}$ as input and transforms it into the final prediction for regression:

$$\hat{\mathbf{Y}} = \mathrm{FC}(\mathbf{Z}). \tag{15}$$

4.5 ⑤ Autocorrelation-Adjusted Distillation Loss

To further enhance the robustness of predictions, we introduce spatial and temporal correlation losses. These losses are designed to reduce sudden spikes or inconsistencies between neighboring nodes and within the same region over nearby time intervals.

The spatial correlation loss is defined as:

$$\mathcal{L}_{\mathrm{spa}}(\tilde{Y}, Y) = \textstyle\sum_{k=1}^{K_r} \left| \hat{y}_t^{(s)} - \hat{y}_t^{(s+k)} \right| \tag{16}$$

The temporal correlation loss is defined as:

$$\mathcal{L}_{\mathrm{tem}}(\tilde{Y}, Y) = \textstyle\sum_{l=-\frac{H}{2}}^{\frac{H}{2}} \left| \hat{y}_t^{(s)} - \hat{y}_{t+l}^{(s)} \right| \tag{17}$$

where $\hat{y}_t^{(s)}$ and $y_t^{(s)}$ denote the predicted value and the corresponding ground truth for region s at time t, respectively. H represents the temporal window size, indicating the number of time slots within the window, and K_r denotes the maximum number of adjacent spatial nodes considered.

Consequently, the overall objective function combines these correlation losses with other loss components to achieve optimal performance:

$$\begin{aligned} \mathcal{L} = \mathcal{L}_{\mathrm{reg}}(\hat{Y}, Y) + \lambda_{\mathrm{tbl}} \mathcal{L}_{\mathrm{tbl}}(\hat{Y}, Y, \tilde{Y}) \\ + \lambda_{\mathrm{KL}} \mathcal{D}_{\mathrm{KL}}(q_\phi(Z|X) || p(Z)) \\ + \lambda_{\mathrm{spa}} \mathcal{L}_{\mathrm{spa}}(\tilde{Y}) + \lambda_{\mathrm{tem}} \mathcal{L}_{\mathrm{tem}}(\tilde{Y}) \end{aligned} \tag{18}$$

where λ_{tbl}, λ_{KL}, λ_{spa}, and λ_{tem} are weights that balance the contributions of the corresponding loss terms.

5 Performance Evaluation

We evaluate the capabilities of our proposed FlowDistill model by addressing seven key research questions (RQs):

- **RQ1:** How do different teacher large language models (LLMs) perform in spatial-temporal adaptation under supervised fine-tuning?
- **RQ2:** How does FlowDistill compare to state-of-the-art baselines in predicting traffic volume, particularly with limited training data (few-shot learning)?
- **RQ3:** How does FlowDistill perform under zero-shot learning scenarios across different cities compared to existing baselines?
- **RQ4:** Can FlowDistill robustly predict traffic across varying temporal and spatial patterns?
- **RQ5:** Does FlowDistill effectively capture spatial and temporal autocorrelations in traffic data?
- **RQ6:** How do different hyperparameter settings affect the performance of FlowDistill?
- **RQ7:** How much does each component of FlowDistill contribute to its overall performance?

5.1 Settings

5.1.1 Dataset Description. In order to evaluate the performance of our proposed method, we conduct experiments using real-world datasets collected from taxi demands in the cities of NYC[1], Chicago[2] and two cities in China [28], as shown in Table 3. These datasets originate from distinct urban environments and exhibit diverse statistical characteristics.

Table 3: The statistic of datasets: Time Span (mm/dd/yy)

Dataset	# Nodes	Sample Rate	Time Span
NYC-Taxi	263	30min	01/01/21 - 12/31/21
CHI-Taxi	77	30min	01/01/21 - 12/31/21
TrafficSH	896	30min	03/05/22- 04/05/22
TrafficHZ	672	30min	03/05/22- 04/05/22

5.1.2 Evaluation Metrics. Similar to [18], we evaluate the accuracy of traffic flow prediction using *Mean Absolute Error (MAE)*, *Root Mean Square Error (RMSE)*, and *Mean Absolute Percentage Error (MAPE)*. These metrics measure the prediction errors of the estimated inflows and outflows from different perspectives. They are defined as:

$$\mathrm{MAE}(y, \hat{y}) = \frac{1}{N \times T} \sum_{i=1}^{N \times T} |\hat{y}_i - y_i| \tag{19}$$

$$\mathrm{RMSE}(y, \hat{y}) = \sqrt{\frac{1}{N \times T} \sum_{i=1}^{N \times T} (\hat{y}_i - y_i)^2} \tag{20}$$

$$\mathrm{MAPE}(y, \hat{y}) = \frac{100\%}{N \times T} \sum_{i=1}^{N \times T} \left| \frac{\hat{y}_i - y_i}{y_i} \right| \tag{21}$$

where y represents the actual inflows/outflows, $\hat{y}$ is the predicted inflows/outflows, N is the total number of nodes, and T is the total number of time intervals.

Furthermore, to quantitatively evaluate the spatial and temporal autocorrelation within the prediction residuals, we utilize Geary's

[1] https://opendata.cityofnewyork.us/
[2] https://data.cityofchicago.org/

Table 4: Performance comparison of supervised finetuning on NYC-Taxi, CHI-Taxi, TrafficHZ and TrafficSH across different prediction horizons. The best results are in bold.

Dataset	LLM	Horizon 3 (90min)			Horizon 6 (180min)			Horizon 12 (360min)			Average		
		MAE	RMSE	MAPE(%)	MAE	RMSE	MAPE(%)	MAE	RMSE	MAPE(%)	MAE	RMSE	MAPE(%)
NYC-Taxi	Llama3	4.94	8.33	32.79	5.06	8.86	38.05	5.40	9.24	46.87	5.09	8.73	40.23
	Qwen3	**4.79**	**8.17**	**33.94**	**5.00**	**8.79**	**34.47**	**5.12**	**9.10**	**37.14**	**5.00**	**8.62**	**39.01**
	GPT2	4.94	8.62	34.21	5.36	9.63	36.76	5.59	9.99	37.50	5.25	9.31	36.35
CHI-Taxi	Llama3	0.59	1.15	**49.60**	0.64	1.26	**53.51**	0.74	1.49	**60.11**	0.65	1.27	**53.64**
	Qwen3	**0.57**	**1.07**	51.86	**0.62**	**1.21**	54.98	**0.69**	**1.36**	60.70	**0.62**	**1.20**	54.90
	GPT2	**0.57**	1.09	51.98	0.64	1.26	55.70	0.71	1.41	58.20	0.63	1.23	54.49
TrafficSH	Llama3	0.14	0.20	452.42	0.14	0.20	495.97	0.16	0.21	550.52	0.15	0.20	496.99
	Qwen3	**0.11**	**0.16**	**350.99**	**0.11**	**0.16**	**342.11**	**0.12**	**0.17**	**338.75**	**0.11**	**0.16**	**340.76**
	GPT2	0.12	0.16	361.16	0.12	0.17	377.83	0.12	0.18	474.11	0.12	0.17	394.84
TrafficHZ	Llama3	0.12	0.18	541.71	0.12	0.18	486.80	0.12	0.19	408.41	0.12	0.18	419.49
	Qwen3	**0.11**	**0.16**	**245.37**	**0.11**	**0.17**	**230.25**	**0.11**	**0.17**	**278.56**	**0.11**	**0.17**	**241.79**
	GPT2	0.11	0.17	376.97	0.11	0.17	369.44	0.12	0.18	330.63	0.11	0.17	339.21

C [7] and the Durbin-Watson statistic [4], respectively. These are defined as:

$$C = \frac{(N-1)\sum_i \sum_j w_{ij}(e_t^{(i)} - e_t^{(j)})^2}{2S_0 \sum_i (e_t^{(i)} - \bar{e}_t)^2} \tag{22}$$

$$DW = \frac{\sum_{t=2}^{n}(e_t^{(i)} - e_{t-1}^{(i)})^2}{\sum_{t=1}^{n}(e_t^{(i)})^2} \tag{23}$$

where $e_t^{(i)} = \hat{y}_t^{(i)} - y_t^{(i)}$ is the residual for region i at time t, $w_{ij} \in \{0, 1\}$ is a binary spatial adjacency weight between the region pair (v_i, v_j), $S_0 = \sum_i \sum_j w_{ij}$, and $\bar{e}_t$ is the mean of the residuals at time t. Both metrics are averaged across all considered dimensions.

5.1.3 Baselines. We compare FlowDistill against six baseline models that have demonstrated strong performance in traffic prediction. We categorize these into graph-based models and knowledge distillation models.

Graph-Based Models: (1) **BigST** [10] is a linear complexity spatial-temporal graph neural network designed for large-scale traffic forecasting. (2) **STSGCN** [20] introduces a synchronous graph convolutional network to model localized spatial-temporal correlations. (3) **ASTGCN** [33] integrates external factors, such as weather and Points of Interest (POIs), into a spatial-temporal graph convolutional network to improve prediction accuracy. (4) **STWA** [3] leverages location-specific and time-varying parameters to capture dynamic traffic patterns.

Knowledge Distillation Models: (5) **ST-MLP** [29] distills knowledge from spatial-temporal graph neural networks (STGNNs) into an MLP, achieving competitive performance with reduced complexity. (6) **EasyST** [23] utilizes a lightweight MLP framework distilled from GNN-based models to enhance scalability and robustness.

5.1.4 Experiment Settings. We conducted experiments on a system equipped with eight NVIDIA H100 GPUs (40GB memory each). For the baseline models, we follow the experimental settings established in OpenCity [17]. We initialize the learning rate at 0.0055 with a decay rate of 0.6 and use a batch size of 80. We use MAE as the regression loss. To capture spatial and temporal correlations, we set the temporal window size to $H = 12$ and the maximum number of adjacent spatial nodes to $K_r = 8$. FlowDistill uses three layers with an embedding size of 64. The spatial context, time-of-day, and day-of-week temporal dimensions are all set to 64. For the teacher LLM adaptation, we employ the LoRA strategy with the following configuration: lora_r = 8, lora_alpha = 32, and lora_dropout = 0.1.

5.2 Supervised Finetuning of Spatial-Temporal Adaptation of LLMs (RQ1)

We evaluated the performance of three teacher LLMs (Llama3, Qwen3, and GPT2) to determine the most effective model for spatial-temporal adaptation. For this evaluation, we used 60% of the data for training and 10% for testing. As shown in Table 4, Qwen3 achieves the lowest MAE, RMSE, and MAPE across all four datasets and prediction horizons (90, 180, and 360 minutes). Consequently, we select Qwen3 as the teacher model for all subsequent FlowDistill experiments.

5.3 Few-Shot Learning (RQ2)

To evaluate the generalization ability of FlowDistill, we compared it against the six baselines across the NYC-Taxi, CHI-Taxi, TrafficHZ, and TrafficSH datasets. Table 5 reports the performance of each model when restricted to only 10% of the available training data. We also record the memory cost of each model. As shown in the table, generally, knowledge distillation-based models exhibit better predictive performance than graph-based models under limited training data. Among them, FlowDistill not only achieves the best prediction accuracy across all datasets but also requires the least memory (0.39 GB).

To simulate varying data availability conditions, we tested the models using training data ratios ranging from 10% to 50%, keeping the testing ratio fixed at 10%. As illustrated in Figures 4 to 7, FlowDistill consistently outperforms all baselines across all training ratios. Notably, on the complex NYC-Taxi dataset (Figure 4), FlowDistill matches the performance of STWA trained on 40% of

Table 5: Model performance comparison using only a 10% training data ratio. The best results are highlighted in bold.

Model	Memory (GB)	NYC-Taxi			CHI-Taxi			TrafficSH			TrafficHZ		
		MAE	RMSE	MAPE(%)	MAE	RMSE	MAPE(%)	MAE	RMSE	MAPE(%)	MAE	RMSE	MAPE(%)
BigST	0.51	18.27	40.46	174.66	5.41	16.56	65.01	3.62	4.34	22.62	1.57	2.06	23.68
STSGCN	1.01	14.79	32.98	647.89	5.49	15.66	76.36	4.04	5.09	16.21	1.29	1.71	20.16
ASTGCN	1.02	8.51	24.46	63.73	5.17	16.30	39.12	2.90	3.50	16.46	1.19	1.58	18.93
STWA	2.08	8.48	19.98	41.29	5.10	16.18	**38.47**	4.50	5.67	**14.98**	1.22	1.63	17.72
ST-MLP	34.10	11.56	25.69	46.76	4.31	12.61	43.12	2.75	3.24	21.60	1.48	1.94	20.12
EasyST	0.41	9.98	22.91	42.45	4.98	13.52	45.12	3.38	4.34	28.96	1.63	2.28	23.01
FlowDistill	**0.39**	**7.22**	**17.02**	**40.75**	**3.46**	**11.69**	41.61	**2.35**	**2.77**	18.47	**0.94**	**1.27**	**14.99**

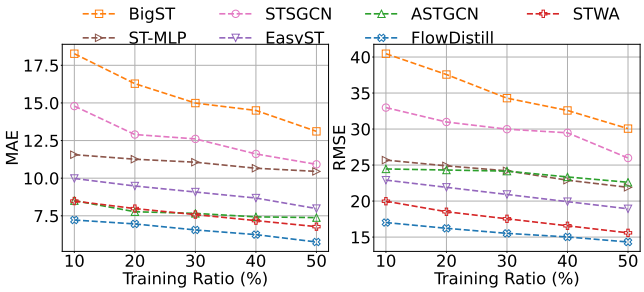


Figure 4: Performance w.r.t training data ratio in NYC-Taxi

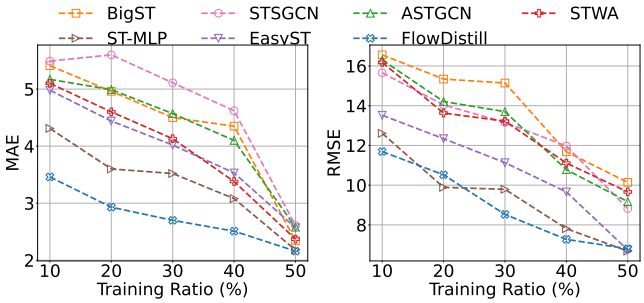


Figure 5: Performance w.r.t training data ratio in CHI-Taxi

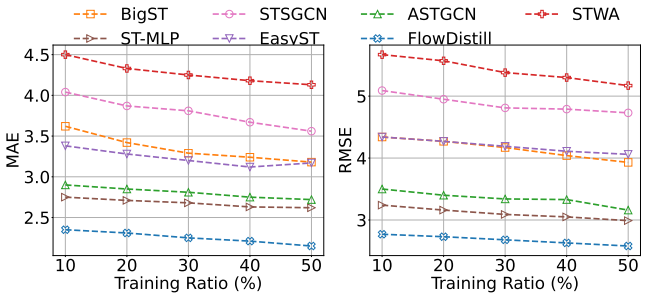


Figure 6: Performance w.r.t training data ratio in TrafficSH

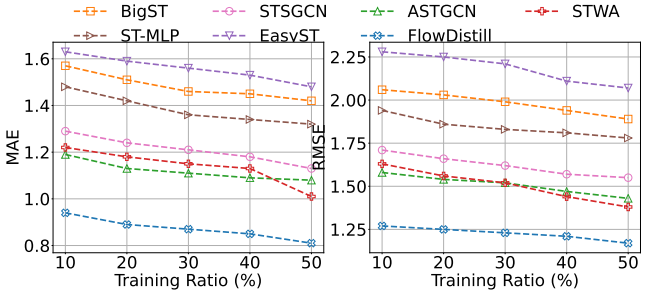


Figure 7: Performance w.r.t training data ratio in TrafficHZ

the data while only using 10% of the training data. For the CHI-Taxi dataset, which contains only 77 nodes and exhibits less complex spatial-temporal dependencies, the performance gap between FlowDistill and the baselines is narrower, yet FlowDistill still maintains its advantage. Figures 6 and 7 further illustrate that on the TrafficSH and TrafficHZ datasets, no other baselines using even 50% training data could match FlowDistill's performance at 10% training data. These results confirm FlowDistill's superior data efficiency and prediction accuracy in few-shot learning scenarios.

5.4 Zero-Shot Learning (RQ3)

To assess the generalizability of FlowDistill in completely unseen environments, we conduct zero-shot learning experiments. We train each model on a source city with the few shot setting and evaluate it on a target city without fine-tuning. We select three transfer settings: a U.S. pair (NYC-Taxi → CHI-Taxi), a China pair (TrafficSH → TrafficHZ), and a cross-country transfer from the U.S. to China (NYC-Taxi → TrafficSH).

As shown in Table 6, we have three observations. (1) Compared with the in-city few-shot results in Table 5, all models degrade under zero-shot transfer, and the degradation is most severe for the third pair (NYC-Taxi → TrafficSH); STSGCN suffers the largest drop. (2) Generally, knowledge-distillation-based models (ST-MLP, EasyST, and FlowDistill) achieve better zero-shot performance than graph-based baselines. (3) The performance gap among models is narrower for the third pair, but FlowDistill maintains a larger advantage over the baselines on the other two pairs, demonstrating the transfer ability of the spatial-temporal instruction prompt for similar layout cities.

Table 6: Zero-shot learning performance comparison across different city pairs (Source City → Target City). The best results are highlighted in bold.

Model	NYC-Taxi → CHI-Taxi			TrafficSH → TrafficHZ			NYC-Taxi → TrafficSH		
	MAE	RMSE	MAPE (%)	MAE	RMSE	MAPE (%)	MAE	RMSE	MAPE (%)
BigST	9.86	29.37	177.82	4.77	4.39	22.38	4.28	5.11	28.73
STSGCN	21.24	62.35	425.57	6.64	6.92	23.71	23.71	67.30	307.91
ASTGCN	9.82	31.83	161.73	2.66	2.56	89.74	4.16	5.39	28.69
STWA	9.86	31.22	121.40	4.91	4.38	33.69	3.67	4.20	22.26
ST-MLP	7.25	18.22	577.79	4.11	3.67	20.67	3.74	4.38	22.70
EasyST	7.96	24.38	140.70	2.83	3.32	17.71	4.14	4.48	27.52
FlowDistill	**3.72**	**12.46**	**40.58**	**1.43**	**2.08**	**12.25**	**3.50**	**3.98**	**19.96**

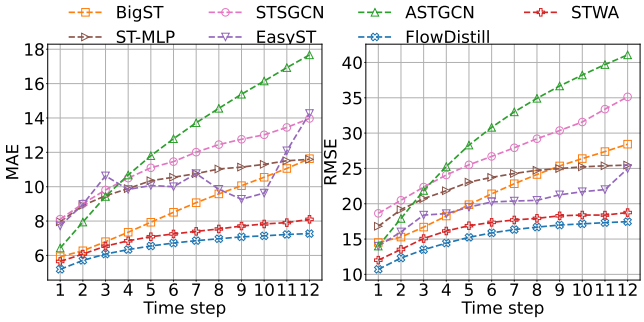


Figure 8: Temporal-based prediction comparison across different horizons in NYC-Taxi.

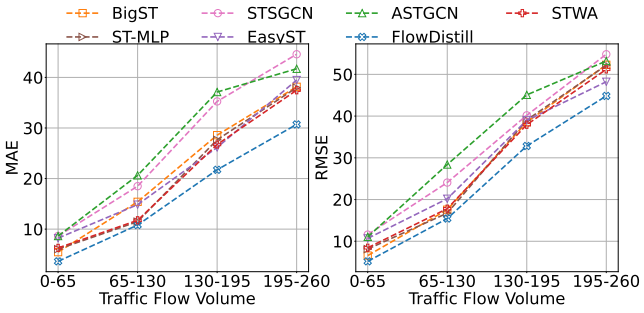


Figure 9: Spatial-based prediction comparison across nodes with varying traffic flow volumes in NYC-Taxi.

5.5 Robustness Analysis (RQ4)

Temporal-based prediction comparison. We examined FlowDistill's performance across 12 consecutive prediction intervals using the NYC-Taxi dataset. As shown in Figure 8, FlowDistill maintains stable and superior performance across both short-term and long-term prediction horizons. This demonstrates its strong generalization ability and temporal robustness compared to baseline models, whose errors accumulate more rapidly over time.

Traffic volume-based prediction comparison. Traffic flow volume varies significantly across different urban areas; nodes with lower volumes typically correspond to residential neighborhoods, whereas high-volume nodes are often located in densely populated commercial zones. We evaluated model performance across nodes with varying traffic flow volumes in NYC-Taxi using a 50% training ratio. Figure 9 shows that FlowDistill consistently outperforms baselines across all volume levels. More importantly, as traffic flow volume increases, FlowDistill maintains a wider performance advantage in both MAE and RMSE, demonstrating its robustness in handling highly dynamic and congested spatial areas.

5.6 Spatial-Temporal Autocorrelation Analysis (RQ5)

Table 7: Comparison of spatial and temporal autocorrelation in NYC-Taxi. The superscripts $^+$ and $^-$ denote the inclusion or exclusion of the spatial-temporal correlation loss (ST Loss). The best results are highlighted in bold.

Model	Geary's C (≈ 1)	Durbin-Watson Test (≈ 2)
BigST	1.12	1.39
STSGCN	1.33	1.16
ASTGCN	0.55	1.12
STWA	0.69	1.40
ST-MLP	1.31	1.20
EasyST	1.18	1.28
Qwen3	0.67	1.24
FlowDistill (ST Loss$^-$)	0.84	1.21
FlowDistill (ST Loss$^+$)	**0.98**	**1.50**

To quantitatively assess the spatial and temporal autocorrelation of the prediction residuals, we employ Geary's C and the Durbin-Watson statistic, respectively. A value of $C = 1$ for Geary's C or $DW = 2$ for the Durbin-Watson statistic indicates that the residuals are uncorrelated. Values of $C < 1$ or $DW < 2$ suggest positive autocorrelation, whereas values of $C > 1$ or $DW > 2$ suggest negative autocorrelation (see Appendix for proofs). Table 7 presents the experimental results using a 50% training and 20% testing split on the NYC-Taxi dataset.

Key observations include: (1) BigST, STWA, and EasyST capture spatial and temporal autocorrelations more effectively than traditional spatial-temporal GNN baselines such as STSGCN and

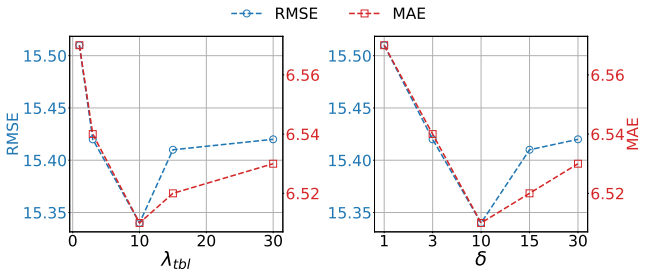


Figure 10: Impact of λ_{tbl} and δ (parameters of the teacher bounded loss) on MAE and RMSE in NYC-Taxi.

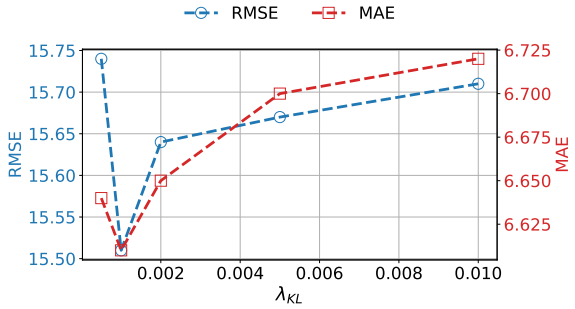


Figure 11: Impact of λ_{KL} (weight of the Information Bottleneck loss) on MAE and RMSE in NYC-Taxi.

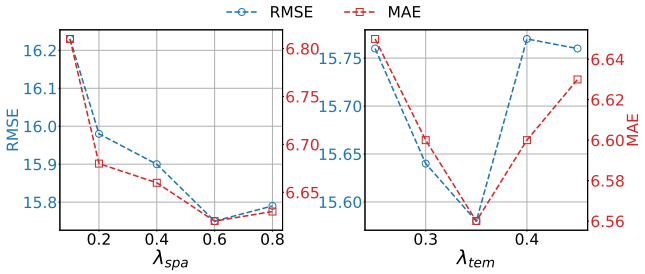


Figure 12: Impact of λ_{spa} and λ_{tem} (weights of the spatial and temporal correlation losses) on MAE and RMSE in NYC-Taxi.

ASTGCN, which exhibit stronger residual autocorrelations. (2) The LLM (Qwen3) struggles significantly to capture dependencies, yielding Geary's C of 0.67 and Durbin-Watson of 1.24, both of which lag behind graph based models like BigST and STWA. (3) Incorporating the ST Loss provides a significant improvement for the student FlowDistill model, improving Geary's C from 0.84 to 0.98 and the Durbin-Watson statistic from 1.21 to 1.50. This proves that our spatial-temporal autocorrelation-aware loss successfully recaptures the dependencies that the LLM misses.

5.7 Hyperparameter Investigation (RQ6)

We conducted experiments to analyze the sensitivity of FlowDistill to five key hyperparameters: (i) δ and λ_{tbl} for the teacher bounded loss, (ii) λ_{KL} for the Information Bottleneck loss, and (iii) λ_{spa} and λ_{tem} for the spatial and temporal correlation losses. During each test, we varied one parameter while keeping the others at their default values. The results are shown in Figures 10, 11, and 12.

Specifically, in Figure 10, we varied λ_{tbl} across $\{0.05, 0.10, 0.15, 0.20, 0.25\}$ and δ across $\{1, 3, 10, 15, 30\}$, identifying the optimal settings as $\lambda_{\text{tbl}} = 0.10$ and $\delta = 10$. In Figure 11, testing λ_{KL} within the range $\{5\times10^{-4}, 1\times10^{-3}, 2\times10^{-3}, 5\times10^{-3}, 1\times10^{-2}\}$ revealed optimal performance at $\lambda_{\text{KL}} = 1 \times 10^{-3}$. Finally, Figure 12 shows the evaluation of λ_{spa} and λ_{tem} over $\{0.2, 0.4, 0.6, 0.8\}$ and $\{0.25, 0.30, 0.35, 0.40, 0.45\}$, respectively. We concluded that the optimal weights are $\lambda_{\text{spa}} = 0.6$ and $\lambda_{\text{tem}} = 0.35$.

5.8 Ablation Study (RQ7)

Table 8: Ablation study evaluating the contribution of each model component in NYC-Taxi. The best results are in bold.

Model variant	MAE	RMSE	MAPE (%)
FlowDistill	**5.89**	**14.39**	**38.17**
w/o-TB	6.89	16.23	74.23
w/o-IB	6.58	15.59	50.12
w/o-SC	7.01	16.89	78.00
w/o-TC	8.01	17.79	89.44

To isolate the contributions of key components in the FlowDistill architecture, we conducted an ablation study. As shown in Table 8, removing any component degrades performance: (1) **Teacher Bounded Loss (w/o-TB):** Its removal increases MAE by 17.0% and RMSE by 12.8%, confirming that bounding the student's output with the teacher's knowledge provides crucial guidance. (2) **Information Bottleneck (w/o-IB):** The performance degradation (MAE increases by 11.7%) confirms its effectiveness in filtering redundant information and improving generalization. (3) **Spatial Correlation Loss (w/o-SC):** Removing this structural constraint increases MAE by 19.0% and RMSE by 17.4%, highlighting its necessity for capturing spatial dependencies between nodes. (4) **Temporal Correlation Loss (w/o-TC):** Its removal causes the most significant drop in accuracy (MAE increases by 36.0% and RMSE by 23.6%), underscoring its critical role in modeling time-series dependencies.

6 Conclusion

In this paper, we present FlowDistill, a novel framework that leverages knowledge distillation from LLMs to enhance traffic flow prediction in both data-scarce and scalable data environments. By incorporating spatial-temporal autocorrelation guidance into the prompt and preserving this autocorrelation during the knowledge distillation process, our framework effectively retains the spatial-temporal dependencies in traffic patterns, leading to more accurate traffic predictions. Our experiments on real-world traffic datasets demonstrate that FlowDistill outperforms traditional graph-based models and state-of-the-art knowledge distillation methods by reducing the the reliance on extensive training data while maintaining high accuracy.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. (2023).

[2] Alexander A. Alemi, Ian Fischer, Joshua V. Dillon, and Kevin Murphy. 2019. Deep Variational Information Bottleneck.

[3] Razvan-Gabriel Cirstea, Bin Yang, Chenjuan Guo, Tung Kieu, and Shirui Pan. 2022. Towards spatio-temporal aware traffic time series forecasting. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 2900–2913.

[4] James Durbin and Geoffrey S Watson. 1950. Testing for serial correlation in least squares regression: I. *Biometrika* 37, 3/4 (1950), 409–428.

[5] Aniekan Essien, Ilias Petrounias, Pedro Sampaio, and Sandra Sampaio. 2021. A deep-learning model for urban traffic flow prediction with traffic events mined from twitter. *World Wide Web* 24, 4 (2021), 1345–1368.

[6] Zheng Fang, Qingqing Long, Guojie Song, and Kunqing Xie. 2021. Spatial-temporal graph ode networks for traffic flow forecasting. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*. 364–373.

[7] R. C. Geary. 1954. The Contiguity Ratio and Statistical Mapping. *The Incorporated Statistician* 5, 3 (1954), 115–146. http://www.jstor.org/stable/2986645

[8] Xusen Guo, Qiming Zhang, Junyue Jiang, Mingxing Peng, Meixin Zhu, and Hao Frank Yang. 2024. Towards explainable traffic flow prediction with large language models. *Communications in Transportation Research* 4 (2024), 100150.

[9] Jindong Han, Weijia Zhang, Hao Liu, Tao Tao, Naiqiang Tan, and Hui Xiong. 2024. Bigst: Linear complexity spatio-temporal graph neural network for traffic forecasting on large-scale road networks. *Proceedings of the VLDB Endowment* 17, 5 (2024), 1081–1090.

[10] Jindong Han, Weijia Zhang, Hao Liu, Tao Tao, Naiqiang Tan, and Hui Xiong. 2024. BigST: Linear Complexity Spatio-Temporal Graph Neural Network for Traffic Forecasting on Large-Scale Road Networks. *Proc. VLDB Endow.* 17, 5 (Jan. 2024), 1081–1090.

[11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.

[12] Geoffrey Hinton. 2015. Distilling the Knowledge in a Neural Network. (2015).

[13] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685* (2021).

[14] Junzhong Ji, Fan Yu, and Minglong Lei. 2023. Self-Supervised Spatiotemporal Graph Neural Networks With Self-Distillation for Traffic Prediction. *IEEE Transactions on Intelligent Transportation Systems* 24, 2 (2023).

[15] Jintao Ke, Hai Yang, Hongyu Zheng, Xiqun Chen, Yitian Jia, Pinghua Gong, and Jieping Ye. 2018. Hexagon-based convolutional neural network for supply-demand forecasting of ride-sourcing services. *IEEE Transactions on Intelligent Transportation Systems* 20, 11 (2018), 4160–4173.

[16] Diederik P Kingma and Max Welling. 2013. Auto-Encoding Variational Bayes.

[17] Zhonghang Li, Long Xia, Lei Shi, Yong Xu, Dawei Yin, and Chao Huang. 2024. Opencity: Open spatio-temporal foundation models for traffic prediction. (2024).

[18] Zhonghang Li, Lianghao Xia, Jiabin Tang, Yong Xu, Lei Shi, Long Xia, Dawei Yin, and Chao Huang. 2024. UrbanGPT: Spatio-Temporal Large Language Models *(KDD '24)*. Association for Computing Machinery, New York, NY, USA, 5351–5362.

[19] Zezhi Shao, Zhao Zhang, Fei Wang, Wei Wei, and Yongjun Xu. 2022. Spatial-Temporal Identity: A Simple yet Effective Baseline for Multivariate Time Series Forecasting. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management* (Atlanta, GA, USA) *(CIKM '22)*. Association for Computing Machinery, New York, NY, USA, 4454–4458. doi:10.1145/3511808.3557702

[20] Chao Song, Youfang Lin, Shengnan Guo, and Huaiyu Wan. 2020. Spatial-temporal synchronous graph convolutional networks: A new framework for spatial-temporal network data forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34. 914–921.

[21] Mingtian Tan, Mike Merrill, Vinayak Gupta, Tim Althoff, and Tom Hartvigsen. 2024. Are language models actually useful for time series forecasting? *Advances in Neural Information Processing Systems* 37 (2024), 60162–60191.

[22] Jiabin Tang, Wei Wei, Lianghao Xia, and Chao Huang. 2024. EasyST: A Simple Framework for Spatio-Temporal Prediction. *arXiv preprint arXiv:2409.06748* (2024).

[23] Jiabin Tang, Wei Wei, Lianghao Xia, and Chao Huang. 2024. EasyST: A Simple Framework for Spatio-Temporal Prediction. *ArXiv* (2024).

[24] Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, Xiaojun Chang, and Chengqi Zhang. 2020. Connecting the dots: Multivariate time series forecasting with graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 753–763.

[25] Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, and Chengqi Zhang. 2019. Graph wavenet for deep spatial-temporal graph modeling. *arXiv preprint arXiv:1906.00121* (2019).

[26] Bing Yu, Haoteng Yin, and Zhanxing Zhu. 2017. Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting. *arXiv preprint arXiv:1709.04875* (2017).

[27] Chenyang Yu, Xinpeng Xie, Yan Huang, and Chenxi Qiu. 2024. Harnessing LLMs for Cross-City OD Flow Prediction. In *Proceedings of the 32nd ACM International Conference on Advances in Geographic Information Systems* (Atlanta, GA, USA) *(SIGSPATIAL '24)*. Association for Computing Machinery, New York, NY, USA, 384–395. doi:10.1145/3678717.3691308

[28] Yuan Yuan, Jingtao Ding, Jie Feng, Depeng Jin, and Yong Li. 2024. UniST: A Prompt-Empowered Universal Model for Urban Spatio-Temporal Prediction. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Barcelona, Spain) *(KDD '24)*. Association for Computing Machinery, New York, NY, USA, 4095–4106. doi:10.1145/3637528.3671662

[29] Junfeng Zhang, Cheng Xie, Hongming Cai, Weiming Shen, and Rui Yang. 2024. Knowledge Distillation-Based Spatio-Temporal MLP Model for Real-Time Traffic Flow Prediction. *IEEE Transactions on Intelligent Transportation Systems* (2024).

[30] Junbo Zhang, Yu Zheng, and Dekang Qi. 2017. Deep spatio-temporal residual networks for citywide crowd flows prediction. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 31.

[31] Chuanpan Zheng, Xiaoliang Fan, Cheng Wang, and Jianzhong Qi. 2020. Gman: A graph multi-attention network for traffic prediction. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34. 1234–1241.

[32] Liangwei Nathan Zheng, Chang George Dong, Wei Emma Zhang, Lin Yue, Miao Xu, Olaf Maennel, and Weitong Chen. 2025. Understanding Why Large Language Models Can Be Ineffective in Time Series Analysis: The Impact of Modality Alignment. arXiv:2410.12326 [cs.LG] https://arxiv.org/abs/2410.12326

[33] Jiawei Zhu, Qiongjie Wang, Chao Tao, Hanhan Deng, Ling Zhao, and Haifeng Li. 2021. AST-GCN: Attribute-augmented spatiotemporal graph convolutional network for traffic forecasting. *Ieee Access* 9 (2021), 35973–35983.

A Proof of Equation (14)

We assume that each dimension of Z is independent and identically distributed (i.i.d.), then The posterior distribution of the MLP can be further elaborated as:

$$\begin{aligned} q_\phi(Z|X) &= \prod_{i=1}^{N\times H_{\text{in}}\times d} q_\phi(z_i|x_i) \\ &= \prod_{i=1}^{N\times H_{\text{in}}\times d} \frac{1}{\sqrt{2\pi}\sigma_i} \exp\left(-\frac{(z_i-\mu_i)^2}{2\sigma_i^2}\right) \end{aligned} \tag{24}$$

Here, z_i represents the i-th independent dimension of the latent variable Z, and x_i represents the corresponding input dimension. The independence assumption implies that $q_\phi(z_i|x_i)$ are independent for all i, where $i \in \{1, 2, \ldots, N \times H_{\text{in}} \times d\}$.

The prior distribution is defined as:

$$\begin{aligned} p(Z) &= \prod_{i=1}^{N\times H_{\text{in}}\times d} p(z_i) \\ &= \prod_{i=1}^{N\times H_{\text{in}}\times d} \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{z_i^2}{2}\right) \end{aligned} \tag{25}$$

Thus, the KL divergence between $q_\phi(Z|X)$ and $p(Z)$ is given by:

$$D_{\text{KL}}(q_\phi(Z|X)||p(Z)) \tag{26}$$

$$= \int q_\phi(Z|X) \log \frac{q_\phi(Z|X)}{p(Z)}\, dZ \tag{27}$$

$$= \int q_\phi(Z|X) \log \frac{\prod_{i=1}^{N\times H_{\text{in}}\times d} q_\phi(z_i|x_i)}{\prod_{i=1}^{N\times H_{\text{in}}\times d} p(z_i)}\, dZ \tag{28}$$

$$= \sum_{i=1}^{N\times H_{\text{in}}\times d} \int \left(\prod_{i=1}^{d} q_\phi(z_i|x_i)\right) \log \frac{q_\phi(z_i|x_i)}{p(z_i)} dZ \tag{29}$$

By the independence assumption, each z_i is independent, and the integral over Z can be factorized into a sum over integrals of z_i:

$$D_{\text{KL}}(q_\phi(Z|X)||p(Z)) \tag{30}$$

$$= \sum_{i=1}^{N\times H_{\text{in}}\times d} \int q_\phi(z_i|x_i) \left(\log \frac{1}{\sigma_i} - \frac{(z_i-\mu_i)^2}{2\sigma_i^2} + \frac{z_i^2}{2}\right) dz_i$$

Since $\int q_\phi(z_i|x_i) dz_i = 1$, the KL divergence becomes:

$$D_{\text{KL}}(q_\phi(Z|X)||p(Z)) = \sum_{i=1}^{N\times H_{\text{in}}\times d} \frac{1}{2}\left(\log \frac{1}{\sigma_i^2} - 1 + \sigma_i^2 + \mu_i^2\right) \tag{31}$$

B Proof of Geary's C Statistic and Durbin-Watson Test

Geary's C is defined as:

$$C = \text{Mean}\left(\frac{(N-1)\sum_i \sum_j w_{ij}(e_t^{(i)} - e_t^{(j)})^2}{2S_0 \sum_i (e_t^{(i)} - \bar{e}_t)^2}\right) \tag{32}$$

we assume residuals have a mean of zero ($\bar{e}_t = 0$) and a constant variance $\text{Var}(e_t^{(i)}) = \sigma^2$. The expected value of the squared difference between two uncorrelated residuals is:

$$E[(e_t^{(i)} - e_t^{(j)})^2] = E[(e_t^{(i)})^2] + E[(e_t^{(j)})^2] - 2E[e_t^{(i)} e_t^{(j)}] \tag{33}$$

When there is no autocorrelation, $e_t^{(i)}$ and $e_t^{(j)}$ are independent for $i \neq j$, so $E[e_t^{(i)} e_t^{(j)}] = E[e_t^{(i)}]E[e_t^{(j)}] = 0$. Thus:

$$E[(e_t^{(i)} - e_t^{(j)})^2] = \sigma^2 + \sigma^2 - 0 = 2\sigma^2 \tag{34}$$

Let $W = \sum_i \sum_j w_{ij}$ be the sum of all weights. The expected value of the numerator is:

$$E[\text{Numerator}] = (N-1) \sum_i \sum_j w_{ij} E[(e_t^{(i)} - e_t^{(j)})^2] = 2\sigma^2 (N-1) S_0 \tag{35}$$

The denominator's term $\sum_i (e_t^{(i)} - \bar{e}_t)^2$ is $(N-1)$ times the sample variance, whose expectation is σ^2. Therefore, the expected value of the denominator is:

$$E[\text{Denominator}] = 2S_0 E[\sum_i (e_t^{(i)} - \bar{e}_t)^2] = 2(N-1) S_0 \sigma^2 \tag{36}$$

The expected value of C is the ratio of the expected numerator to the expected denominator:

$$E[C] \approx \frac{2\sigma^2 (N-1) S_0}{2S_0 (N-1)\sigma^2} = 1 \tag{37}$$

With this condition on the weight matrix, a value of $C \approx 1$ indicates no significant spatial autocorrelation.

The Durbin-Watson statistic (DW) is defined as:

$$DW = \text{Mean}\left(\frac{\sum_{t=2}^{n} (e_t^{(i)} - e_{t-1}^{(i)})^2}{\sum_{t=1}^{n} (e_t^{(i)})^2}\right) \tag{38}$$

we first expand the numerator: $\sum_{t=2}^{n} (e_t^{(i)} - e_{t-1}^{(i)})^2 = \sum_{t=2}^{n} (e_t^{(i)})^2 + \sum_{t=2}^{n} (e_{t-1}^{(i)})^2 - 2\sum_{t=2}^{n} e_t^{(i)} e_{t-1}^{(i)}$

For a sufficiently large sample size n, the following approximations hold: $\sum_{t=2}^{n} (e_t^{(i)})^2 \approx \sum_{t=1}^{n} (e_t^{(i)})^2$ and $\sum_{t=2}^{n} (e_{t-1}^{(i)})^2 \approx \sum_{t=1}^{n} (e_t^{(i)})^2$

When there is no autocorrelation, the sample covariance between adjacent residuals approaches zero.

Therefore:

$$\sum_{t=2}^{n} e_t^{(i)} e_{t-1}^{(i)} \approx 0 \tag{39}$$

Substituting these approximations back into the expanded numerator yields: $\sum_{t=2}^{n} (e_t^{(i)} - e_{t-1}^{(i)})^2 \approx \sum_{t=1}^{n} (e_t^{(i)})^2 + \sum_{t=1}^{n} (e_t^{(i)})^2 - 0 = 2\sum_{t=1}^{n} (e_t^{(i)})^2$

Finally, substituting this result into the DW formula gives:

$$DW \approx \frac{2\sum_{t=1}^{n} (e_t^{(i)})^2}{\sum_{t=1}^{n} (e_t^{(i)})^2} = 2 \tag{40}$$

Thus, a DW value of approximately 2 indicates no significant serial correlation in the residuals.